

Case-Specific AI System

Simple Technical Concept & Architecture Document

1. Main Idea

The app will allow a user to create multiple cases. Every case will have its own folder, documents, notes, and a separate AI knowledge space. The AI for one case will only use information belonging to that case. It will not use or mix information from another case.

Simple Example

Case 001 → Case Folder → Documents → Case AI 001

Case 002 → Case Folder → Documents → Case AI 002

Case 003 → Case Folder → Documents → Case AI 003

2. Important Clarification

We do not need to create or train a completely new AI model every time a new case is created. Instead, the app can use the same powerful AI model while giving each case its own isolated knowledge base and memory.

This makes the system faster, cheaper, easier to maintain, and easier to scale.

3. How Case Creation Works

Step 1: User creates a new case.

Step 2: The system generates a unique Case ID.

Step 3: A separate case folder/space is created.

Step 4: Documents, images, notes, and other case information are stored with that Case ID.

Step 5: A separate knowledge base is created for that case.

Step 6: The Case AI can now search only that case's information.

4. Example Scenario

Suppose the user creates Case 001 at 12:00 PM. The system creates Case AI 001 and connects it only to Case 001 data.

After 1.5 hours, the user creates Case 002. The system creates a new Case ID and a separate knowledge space for Case 002.

If the user asks Case AI 001 a question, the AI searches only Case 001 data. Case 002 information is not included in the search. The same rule applies in the opposite direction.

5. System Architecture

Layer	Purpose
Flutter App	User interface for creating cases, uploading files, and chatting with Case AI.
Backend	Handles users, cases, permissions, AI requests, file processing, and security.

Supabase PostgreSQL	Stores users, cases, metadata, notes, and other structured data.
Supabase Storage	Stores case documents, images, PDFs, and other files.
Vector Search / Knowledge Base	Stores searchable representations of case documents.
LLM / AI Model	Reads the relevant case information and generates the answer.

6. Case Isolation - The Most Important Part

Every piece of case data should be connected to a Case ID. For example:

```
User ID: U123
Case ID: CASE001
Document: evidence.pdf
```

When the user asks a question inside CASE001, the backend should search only records where Case ID = CASE001.

This is much safer than simply telling the AI in a prompt: 'Do not use other cases.' The database and backend should enforce the separation.

7. Question-to-Answer Flow

```
User Question
↓
Selected Case ID
↓
Search only that Case ID's knowledge base
↓
Find relevant documents / notes
↓
Send only relevant case information to the AI
↓
AI generates the answer
↓
Answer shown to the user
```

8. Example of Isolation

User asks	Allowed data	Not allowed
Question about Case 001	Case 001 documents, notes, images	Case 002 / Case 003 data
Question about Case 002	Case 002 documents, notes, images	Case 001 / Case 003 data

9. Recommended Technology Stack

Component	Recommended Option
Mobile App	Flutter
Backend	FastAPI (Python) or Node.js
Database	Supabase PostgreSQL
File Storage	Supabase Storage
AI	LLM API
Knowledge Search	Vector embeddings + vector search
Security	User ID + Case ID + Row Level Security

10. Why This Approach Is Better

- **Lower cost:** We do not need to train a new AI model for every case.
- **Better security:** Case-level access rules can prevent cross-case data access.
- **Less data mixing:** Each case has its own searchable knowledge space.
- **Easy to scale:** New cases can be created using the same architecture.
- **Easy to update:** New documents can be added to an existing case without rebuilding the whole AI system.

11. Important Security Rules

- Every case must have a unique Case ID.
- Every document, note, image, and vector record must be linked to its Case ID.
- The backend must verify both User ID and Case ID before returning data.
- Users must not be allowed to access another user's case data.
- Use database-level security such as Row Level Security where supported.
- Do not depend only on an AI prompt to keep cases separated.
- Log important access and AI actions for auditing.

12. Final Concept

The final system can be thought of as **one AI engine with many independent case memories**.

The AI model can be shared, but the knowledge and access for every case remain separate. When a user selects a case, the application passes that Case ID through the backend, retrieves only that case's information, and gives the relevant information to the AI.

In short: New Case → New Case ID → New Case Folder → New Knowledge Space → Case-specific AI experience.

This document describes the proposed architecture at a high level. The exact implementation can be finalized after deciding the app's full features, AI provider, document types, and security requirements.